



UserFS

A User-Space Filesystem Library

innovatrix Team

Design walkthrough and testing report



Agenda

1. The task & design goals

2. On-disk layout

3. Allocation strategy

4. Namespace, links & metadata

5. Journal, mmap & synchronization

6. Testing & verification

7. Results, limits & team



The Task



What is UserFS?

- A C library that implements a small filesystem entirely in user space.
- No kernel modifications, no kernel module, and no FUSE.
- One regular Linux file is one complete, self-contained disk image.
- Programs link libuserfs.a directly and call a POSIX-like API (`ufs_open`, `ufs_read`, `ufs_mkdir`, `ufs_stat`, ...).



Key Design Numbers

Property	Value
Image size	16 MiB (16,777,216 bytes)
Block size	512 bytes
Total blocks	32,768
Maximum inodes	512
Inode size	128 bytes
Directory-entry size	64 bytes
Open descriptors / process	32
Active mappings / process	32
Maximum name / path	31 chars / 255 chars

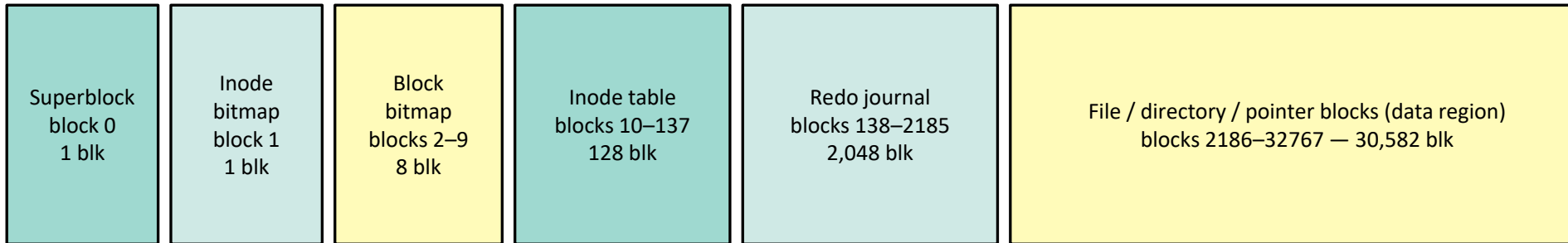
Fixed geometry keeps validation and block addressing simple: `ufs_format()` accepts only 16 MiB, and `ufs_mount()` rejects any image whose size or superblock geometry does not match.



On-Disk Layout



One File, Six Regions



32,768 total blocks × 512 bytes = 16 MiB. The block bitmap has one bit per block; blocks 0–2185 are marked reserved at format time, so the allocator can only ever return blocks in the data region.

- Inode bitmap: one bit per inode; inode 0 is reserved for the root directory — 511 initially free.
- The root inode starts at size 0 with no data block; its first directory block is allocated on the first entry.



Superblock & Inodes

Superblock (512 bytes)

- Magic 0x55465331 (“UFS1”) + format version
- Block size, total blocks, every region's start/length
- Total/free inode and data-block counters
- Root inode number
- Journal location & sequence
- Clean / dirty mount state
- Feature flags: journaling, metadata, links

Inode (128 bytes, 4 per table block)

- Type (file / directory / symlink), flags
- Permission mode, owner UID, owner GID
- Persistent hard-link count
- 64-bit size + logical data-block count
- 8 direct pointers + 1 single-indirect + 1 double-indirect
- Generation number — detects stale descriptors/mappings after a slot is reused
- atime / mtime / ctime

Every unused pointer is UFS_INVALID_BLOCK (0xFFFFFFFF), never 0 — block 0 is a real block, occupied by the superblock.

Allocation Strategy, and journal



Inode & Block Allocation

Inode allocation

- Scan the inode bitmap for a clear bit
- Reserve it, initialize with invalid pointers
- Decrement free_inodes
- Releasing clears the bit and makes the slot reusable
- **Inode 0 is never released**

Block allocation (first-fit)

- Scan the block bitmap from block 2186 for the first clear bit
- Mark it allocated, zero it before use
- Decrement free_blocks
- Freeing reverses the bitmap bit and counters
- **Blocks below 2186 can never be returned — enforced structurally, not just by convention**

Compile-time `_Static_assert` checks pin the superblock, inode, timestamp and directory-entry structures to their intended sizes — the on-disk layout can't silently drift from the design.

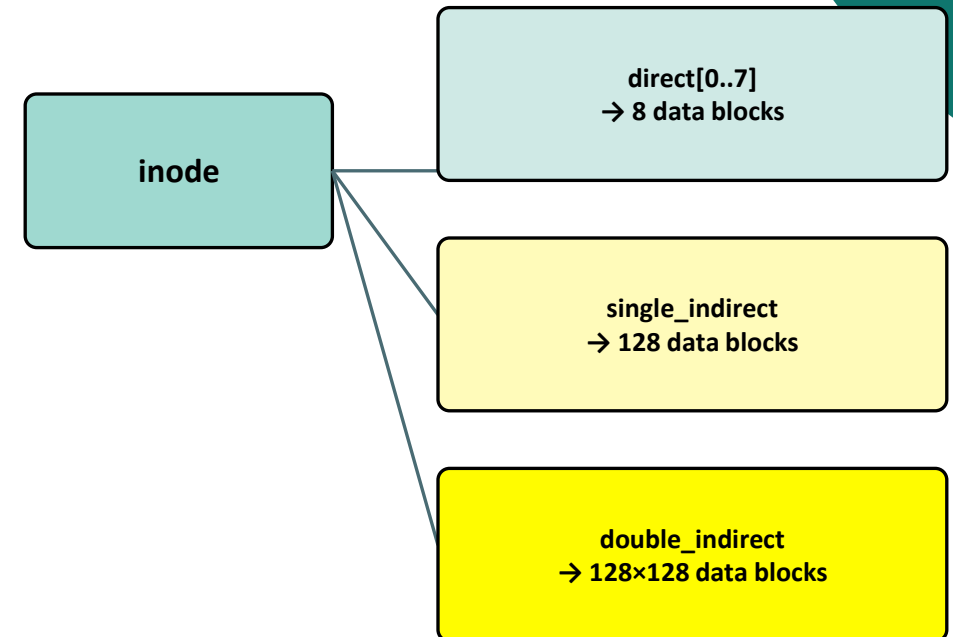
Logical → Physical Block Addressing

Logical range	Mechanism	Capacity
0 – 7	8 direct pointers	8 blocks
8 – 135	1 single-indirect block	128 blocks
136 – 16,519	double-indirect tree	16,384 blocks

Theoretical maximum file size

$(8 + 128 + 128 \times 128) \times 512 = 8,458,240$ bytes

Each pointer is 32 bits, so one 512-byte pointer block stores 128 addresses. Pointer/index blocks are themselves data-region allocations, tracked by the block bitmap.



Directory Entries

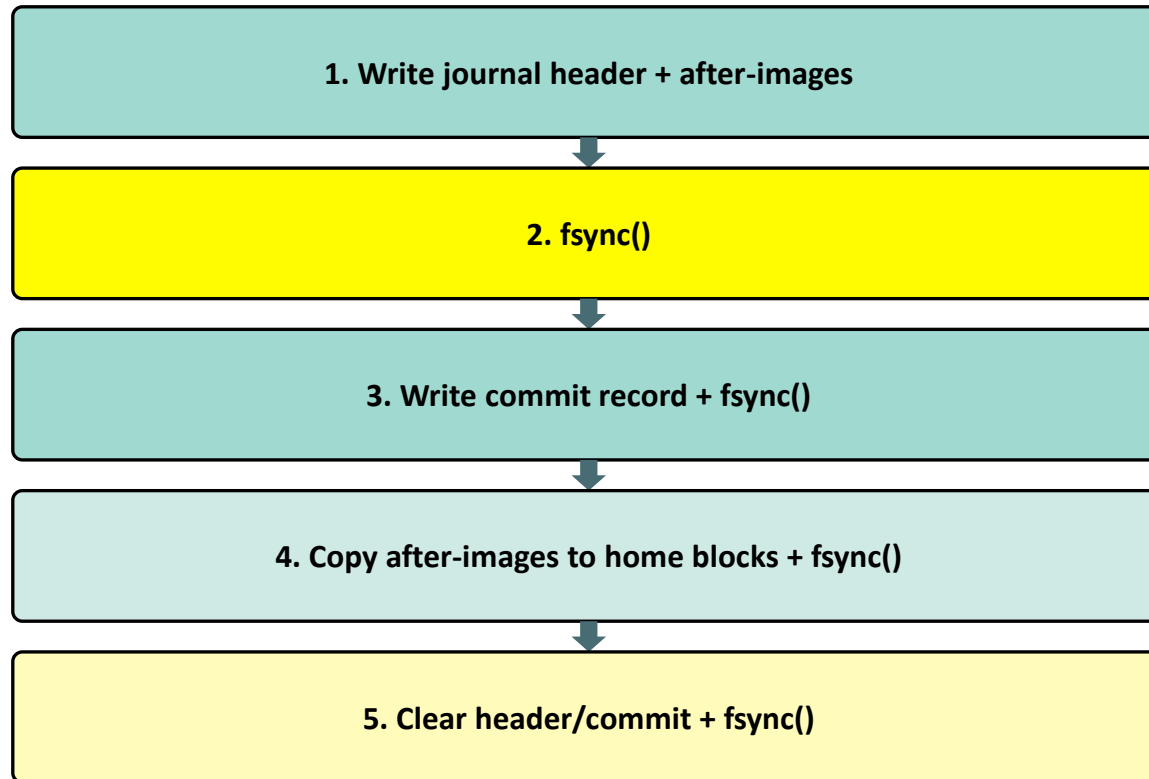
- A directory's data blocks hold fixed 64-byte records: used flag, inode number, object type, and a null-terminated name (≤ 31 chars).
- One 512-byte block holds 8 entries.
- **The current implementation uses only the inode's 8 direct pointers for directory data → a hard cap of 64 entries per directory.**
- Deleted entries are reusable slots.
- Directory size is the allocated capacity in bytes (a multiple of 512), not the sum of name lengths.

One directory block (512 bytes)

entry 0	—	used		inode#		type		name
entry 1	—	used		inode#		type		name
entry 2	—	used		inode#		type		name
entry 3	—	used		inode#		type		name
entry 4	—	used		inode#		type		name
entry 5	—	used		inode#		type		name
entry 6	—	used		inode#		type		name
entry 7	—	used		inode#		type		name



Redo Journal & Crash Recovery



- Physical-block after-images; up to 124 unique target blocks per transaction.
- Mount checks header, commit sequence, target ranges and payload checksum.
- A valid, fully committed transaction is replayed; an incomplete or checksum-invalid one is discarded.
- **Full-block replay is idempotent — safe even if a crash interrupts an earlier replay.**

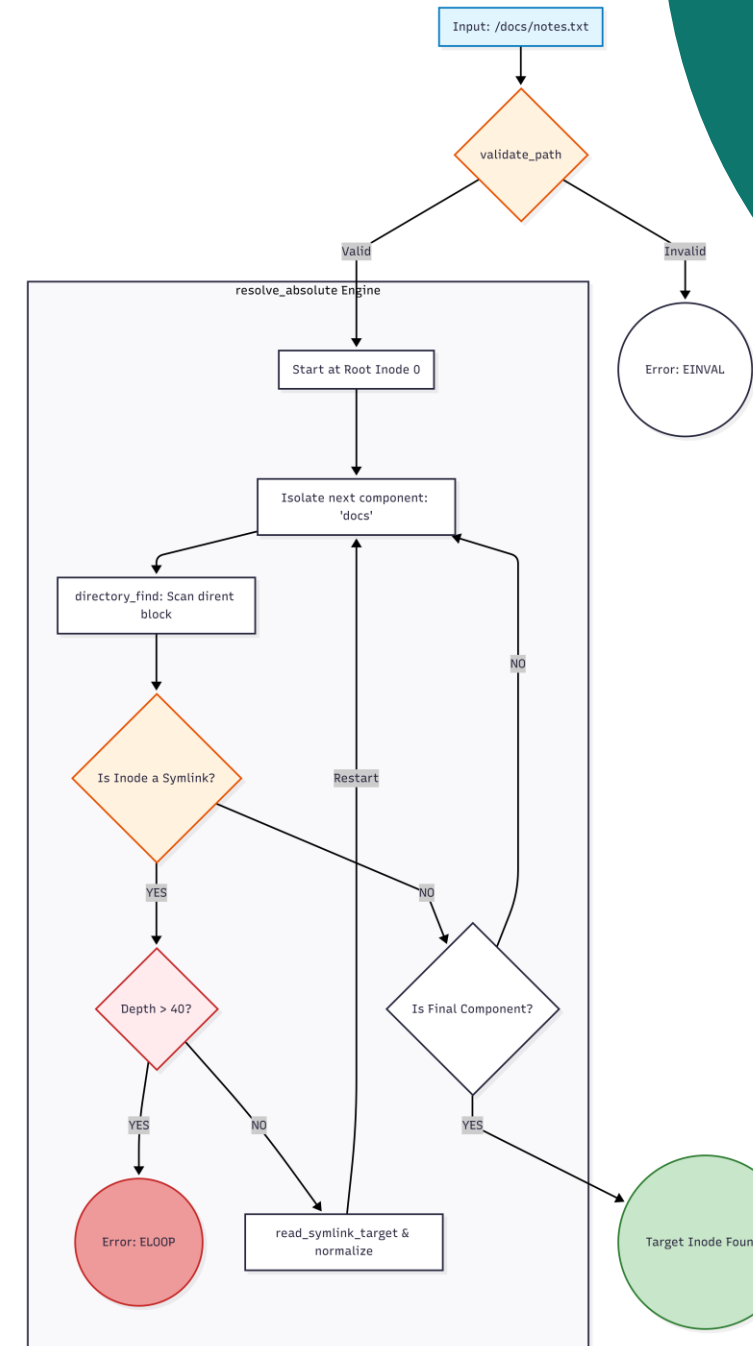


Namespace, Links & Metadata



Namespace, Path Resolution & Traversal Architecture

- **The Traversal Pipeline:** `resolve_path()` and `resolve_absolute()` translate absolute string paths into target Inode numbers iteratively component-by-component.
- **Directory Table Scanning:** `directory_find()` scans fixed-capacity 64-byte dirent blocks, mapping string names to inode numbers.
- **Symbolic Link Redirection:** Intercepts symlinks mid-traversal, extracts target text, normalizes relative paths via `normalize_absolute_path()`, and restarts resolution.
- **Defensive Safety Limits:** Strict absolute path enforcement, no directory hard-linking (prevents DAG cyclic loops), and a 40-expansion ceiling on symlinks to safely trigger ELOOP.



Path Resolution & Namespace Rules

- The public API takes absolute paths only; path/component length limits are enforced and malformed components are rejected.
- “/” resolves directly to inode 0; resolution walks each directory entry in turn.
- Traversing through a non-directory returns ENOTDIR.
- **Duplicate names → EEXIST. Missing paths → ENOENT. Non-empty directory removal → ENOTEMPTY.**
- / can never be removed; an open or actively mapped inode cannot be removed.
- Allocation failures never leave a partially visible object in the namespace.



Hard Links & Symbolic Links

Hard links

- `ufs_link()` adds another directory entry pointing at the same inode
- `link_count` increments on link, decrements on unlink
- Data and inode are freed only once `link_count` reaches zero
- Directory hard links are prohibited — prevents namespace cycles

Symbolic links

- Type `UFS_TYPE_SYMLINK`; target text stored in a UserFS data block
- Targets may be absolute or relative to the link's parent
- Normal resolution follows links; `ufs_lstat()/ufs_readlink()` inspect the link itself
- At most 40 expansions — excess or cyclic chains return `ELOOP`

Both were explicitly listed as optional simplifications in the assignment — implemented anyway, with dedicated loop-safety and remount-persistence tests.



Path Resolution & Namespace Rules

- The public API takes absolute paths only; path/component length limits are enforced and malformed components are rejected.
- “/” resolves directly to inode 0; resolution walks each directory entry in turn.
- Traversing through a non-directory returns ENOTDIR.
- **Duplicate names → EEXIST. Missing paths → ENOENT. Non-empty directory removal → ENOTEMPTY.**
- / can never be removed; an open or actively mapped inode cannot be removed.



File Descriptors & I/O

The descriptor table

- 32 slots per process — in-memory only, not part of the on-disk format
- Each slot: active flag, inode number + generation, independent byte offset, access/append flags
- Generation check rejects a descriptor left pointing at a reused, freed inode
- A full table returns EMFILE on `ufs_open()`

Core operations

- `ufs_open()` validates path, type, flags and requested permissions
- `ufs_read()` stops at EOF, crosses block boundaries, advances the offset
- `ufs_write()` overwrites or extends, allocates blocks as needed, zero-fills a seek-created hole, and advances the offset
- **append mode re-resolves EOF on every write, not just at open time**
- `ufs_seek()` supports `SEEK_SET` / `SEEK_CUR` / `SEEK_END`; seeking past EOF never allocates or resizes
- `ufs_truncate()` grows with zeros or shrinks while releasing blocks



mmap & Synchronization

beyond the required scope



Copy-Backed mmap

Copy-backed memory mapping

- `ufs_mmap()` copies the requested range into anonymous Linux memory
- A private mapping never writes back
- `ufs_msync()` copies a writable shared mapping back to the inode
- `ufs_munmap()` auto-syncs a writable shared mapping first
- Inode-generation check stops a stale mapping from writing a reused inode
- Unmount/unlink with active mappings → EBUSY

Permissions, Ownership & Metadata

- New files: 0666 & ~umask. New directories: 0777 & ~umask. Default umask: 0022.
- Access checks: owner bits → group bits (including supplementary groups) → other bits.
- *Effective UID 0 bypasses permission checks — a documented project simplification.*
- Public API: ufs_chmod(), ufs_chown(), ufs_access(), ufs_utimens().
- Creation initializes all three timestamps; writes/truncation update mtime + ctime; reads apply a reltime-style atime update.



Synchronization

Synchronization model

- Process-local read/write lock plus a whole-image advisory fcntl lock
- The mapping table has its own mutex
- One global mounted context per process
- A foundation for coordinated access — not a claim of full multi-threaded / multi-process POSIX semantics

Important Invariants

Block 0 always contains a valid Version 2 superblock

Inode 0 always represents / and is never freed

Blocks below 2186 are never returned by the data allocator

A set bitmap bit always means “allocated / reserved”

$\text{block_count} = \text{ceil}(\text{size} / 512)$; pointer blocks are additional metadata

Successful operations remain visible after unmount + remount

Newly allocated blocks and exposed ranges always read as zero

ENOSPC never leaves a partially created namespace object

8 of the 14 invariants documented in DESIGN.md §12 — the full list is enforced by the test suite.



Testing & Verification



Testing Strategy

Test file	Main coverage
test_integration.c	Lifecycle, namespace, I/O, seek, truncate, errors, persistence
test_storage_v2.c	Direct, single-indirect and double-indirect allocation
test_journal.c	Commit, replay, incomplete transactions, persistence
test_metadata.c	Permissions, ownership, timestamps, metadata persistence
test_links.c	Hard links, symbolic links, loops, remount behavior
test_mmap.c	Shared/private mappings, synchronization, errors, persistence

Bonus coverage beyond the required scope: `test_permissions_namespace.c`, `test_crash_recovery.c`, `test_threads.c`, `test_processes.c`.

Compiled with `-std=c11 -Wall -Wextra -Werror -pedantic`, and additionally run under `AddressSanitizer` and `UndefinedBehaviorSanitizer`.



Verified Results

302 / 0

checks passed / failed, across 10
test binaries

0

compiler warnings under -Wall -
Wextra -Werror -pedantic

Clean

AddressSanitizer +
UndefinedBehaviorSanitizer runs

100%

of tested persistence scenarios
passed after unmount + remount

- make test: test_integration (87), test_storage_v2 (2), test_journal (3), test_metadata (2), test_links (63), test_mmap (55), test_permissions_namespace (29), test_crash_recovery (12) — all pass.
- make concurrency: test_threads (35), test_processes (14) — all pass.
- make sanitize: integration, links and mmap suites re-run under ASan + UBSan — no sanitizer reports.



Shell & Interactive Demo

An interactive REPL — `apps/userfs_shell.c` — built directly on the library, used to manually exercise every API path and to demo the filesystem live.

Filesystem commands

- `format` · `mount` · `unmount` · `pwd` · `cd`
- `mkdir` · `create` / `touch` · `rmdir` · `rm`
- `write` · `append` · `cat` · `truncate`
- `ls` · `stat` · `lstat`
- `ln` · `readlink` · `chmod`

Debug & inspection commands

- `super` — dump the superblock
- `bitmap` — inode/block bitmap occupancy
- `inode <n>` — raw inode fields
- `map <path>` — logical→physical block mapping
- `show <path>` — raw block/content dump
- `features` · `help` · `quit` / `exit`

The same binary backs today's live demo: format an image, create nested files, unmount/remount, and show the data survived.

TASK.md — Met and Exceeded

Required

- ✓ C filesystem implementation
- ✓ DESIGN.md — layout, allocation, invariants
- ✓ Design & testing slides (this deck)
- ✓ format / mount / unmount / mkdir / rmdir / listdir
- ✓ create / open / close / read / write / seek / truncate / unlink
- ✓ ufs_stat() with type, size & metadata
- ✓ Persistence across unmount + remount, nested directories
- ✓ errno-based error handling (ENOENT, EEXIST, ENOSPC, ...)

Implemented beyond the “simplifications” list

- ✓ Symbolic links + a public hard-link API
- ✓ Permissions, ownership, timestamps
- ✓ Redo journaling with crash recovery
- ✓ Copy-backed mmap / msync / munmap
- ✓ Tested multi-thread synchronization
- ✓ Tested multi-process synchronization scenarios

“Simplifications” in TASK.md were explicitly optional — “no pressure ;)”. The team implemented tested versions of these optional extensions, each with dedicated tests.



Team

Mariam

Lifecycle, shared layout & synchronization

userfs.h / ufs_internal.h ownership, format/mount/unmount, superblock validation, sync primitives

Arwa

Storage, allocation & journal

bitmaps, inode table I/O, block mapping, truncate, redo transactions & recovery

Yasmeen

Paths, namespace & links

path resolution, directories, hard/symbolic links, mkdir/rmdir/create/unlink/listdir/stat

Adn

Descriptors, file I/O, mmap, shell & integration

open/close/read/write/seek/truncate, mmap/msync/munmap, descriptor table, shell, final merge

Jana

Metadata, QA & documentation

permissions/ownership/timestamps, integration & persistence tests, DESIGN.md



Thank You

Innovatrix team ;)

